DummyBank

Simulation Platform

Full-Stack Banking Engine with Predictive Intelligence

Submitted By:

Mithil Kale -1262254135
Diya Varma – 1262252717
Arya Agrawal – 1262252003
Aayush Khandelwal - 1262252807

[Institution / University Name] ·
CSE(AI/DS)/MIT WPU

1. Abstract

NeoBank is a full-stack banking simulation platform combining a React frontend, a Node.js/Express API server, and a high-performance C banking engine. It replicates modern digital banking — account management, deposits, withdrawals, transfers, transaction history, and spending analytics — within a responsive, mobile-first dashboard.

Two intelligent features differentiate NeoBank from conventional simulations:

- Safe-to-Spend Indicator — deducts monthly budget and scheduled transfers from the live balance to give users a forward-looking view of financial health, not just a historical snapshot.
- Impulse Purchase Classifier — a lightweight ML-inspired model that classifies spending as Impulse, Necessary, or Frequent, triggering real-time pre-transaction warning banners for flagged purchases.

★ *NeoBank shifts the paradigm from retrospective ledger to prospective advisor — telling users what they can safely spend, and warning them before they overspend.*

2. Introduction & Objectives

Most academic banking simulations stop at basic account management — they show a balance and call it done. NeoBank was built to challenge that norm by addressing three problems: (1) developers lack a realistic end-to-end banking environment to learn full-stack skills; (2) existing simulations are passive ledgers without financial intelligence; (3) users make poor financial decisions because tools show the past, not the future.

Functional Objectives

Objective	Description	Priority
Account Management	Create, list, search, and delete bank accounts	Critical
Transactions	Deposits, withdrawals, and P2P transfers with validation	Critical
Transaction History	Per-account timestamped transaction logs	Critical
Budget Controls	Set monthly budget and track spending progress	High
Safe-to-Spend	Compute and display predictive spendable balance	High — USP
Impulse Classifier	Identify and flag impulsive spending pre-transaction	High — USP
Statements	Date-range filtered account statement views	Medium

Non-Functional Objectives

- Performance: C engine executes core operations in under 50ms; API round-trip under 200ms.
- Responsiveness: Full mobile-first design tested from 320px to 2560px.
- Maintainability: Modular three-tier architecture — UI, API, and engine independently changeable.
- Usability: All core flows completable in 3 clicks with zero onboarding documentation.

3. System Architecture & Technology Stack

NeoBank is structured as a three-tier application. The React frontend communicates with the Express API over HTTP/REST. The API manages a persistent child process running the compiled C banking engine, forwarding commands via stdin and reading structured responses from stdout.

Architecture Overview

```
React Frontend  (Port 3000)
  Dashboard · Accounts · Transactions · Analytics · Dark Mode
          |  HTTP REST (JSON)
Node.js / Express API  (Port 5000)
  Route Handlers · C Process Manager · JSON Parser
          |  stdin / stdout (text protocol)
C Banking Engine  (bank.c)
  Command Parser · Business Rules · File I/O
          |  Binary Read/Write
accounts.dat  (Binary Flat File)
```

Technology Stack

Layer	Technology	Role
Frontend	React 18 + JavaScript	UI components, routing, state management
Styling	CSS3 (custom)	Responsive layout, dark mode, animations
API Server	Node.js + Express	REST endpoints, C process management
Banking Engine	C (ISO C99)	Core business logic, data operations
Persistence	Binary flat file (.dat)	Account & transaction storage
Build	Makefile + Bash (start.sh)	Compile C engine, launch all services

4. Predictive Intelligence — The NeoBank USP

4.1 Safe-to-Spend Indicator

Most banking apps answer: 'What is my balance?' NeoBank answers: 'What can I safely spend right now?' The Safe-to-Spend Indicator computes a forward-adjusted balance in real time:

```
Safe-to-Spend  =  Total Balance
                - Monthly Budget (user-defined)
                - Σ Scheduled / Recurring Transfers
                - Rolling spend for current month

< 0%        →  Danger Zone   (red)
< 20% bal   →  Caution       (amber)
≥ 20% bal   →  Healthy       (green)
```

Example: Balance ₹12,400 | Budget ₹3,000 | EMI ₹2,500 | MTD Spend ₹1,800 → Safe-to-Spend: ₹5,100 (Healthy)

4.2 Impulse Purchase Classifier

A lightweight, pattern-based model analyses transaction history by category. It is designed to be transparent and explainable — not a black box.

- Step 1: Compute category_ratio = category_spend / total_spend (last 30 days)
- Step 2: Compute category_frequency = count of transactions in this category
- Step 3: Apply threshold matrix → classify as IMPULSE, FREQUENT, or NECESSARY
- Step 4: If IMPULSE — show pre-transaction warning banner with balance impact

Category	Impulse Trigger	Frequent Trigger
Entertainment	ratio > 30%, freq ≥ 5	freq ≥ 8, ratio < 15%
Shopping	ratio > 35%, freq ≥ 4	freq ≥ 10, ratio < 20%
Dining / Food	ratio > 25%, freq ≥ 6	freq ≥ 12, ratio < 20%
Travel	ratio > 20%, freq ≥ 3	freq ≥ 6, ratio < 10%
Utilities / Bills	Never — always NECESSARY	—

5. Backend Design — API & C Engine

5.1 REST Endpoints

Method + Endpoint	C Command	Description
POST /api/accounts	CREATE name type balance	Create new account
GET /api/accounts	LIST	Retrieve all accounts
GET /api/accounts/:id	GET id	Get account by ID
DELETE /api/accounts/:id	DELETE id	Remove account
POST /api/transactions/deposit	DEPOSIT id amount	Credit an account
POST /api/transactions/withdraw	WITHDRAW id amount	Debit (validated)
GET /api/transactions/:id	HISTORY id	Transaction log
GET /api/health	(internal)	Server + engine status

5.2 C Engine Design (bank.c — ISO C99)

The C engine is the heart of NeoBank. Express communicates with it via a line-based stdin/stdout protocol — each command is a single newline-terminated line; responses are structured text parsed into JSON by the API layer.

- Data structures: Account struct {int id, char name[64], char type[20], double balance, time_t created_at} and Transaction struct {int txn_id, int account_id, char type[12], double amount, double balance_after, time_t timestamp, char note[128]}
- Balance validation: withdrawals rejected if amount > current balance; floating-point precision handled explicitly.
- ID generation: auto-incrementing integers with collision checks on load.
- Atomicity: read-modify-write pattern on file writes; partial writes detected on reload via CRC-style validation.
- Command loop: reads from stdin indefinitely; Express writes commands as needed.

5.3 Data Storage

Property	Detail
File	accounts.dat (binary flat file)
Format	Fixed-size struct records written with fwrite() / read with fread()
Account Record	Fixed 256-byte block: ID, name, type, balance, timestamp
Transaction Log	Appended variable-length records after account section
Load Strategy	Full file read on engine start; indexes built in memory
Write Strategy	Seek to record offset + overwrite for updates; append for new records
Recovery	CRC-style validation on startup; corrupted records skipped

6. Frontend Design

The NeoBank UI is built to feel like a real neo-bank product. All design decisions are competition-ready:

Screen / Component	Key Features
Dashboard	Safe-to-Spend indicator (colour-coded), total balance, recent transactions, budget progress ring, server status dot, quick-action buttons
Account Screens	Card-based list with type badges, balance, creation date; detail view with deposit/withdraw/delete actions
Transaction Form	Real-time validation, amount input, account selector, impulse warning banner when triggered
Transaction History	Chronological log with credit/debit colour coding, category icons, amount, timestamp
Budget & Statements	Monthly budget ring chart, category breakdown bar, date-filtered statement view
Dark Mode	Full system-level dark theme toggle persisted to localStorage
Mobile Layout	Hamburger navigation, stacked cards, touch-optimised inputs; tested from 320px
Notifications	Toast-style in-app alerts for transaction confirmations, errors, and warnings

7. Testing & Validation

Test Case	Expected Result	Result
TC-01: Create account with ₹1,000	Account ID returned, balance = 1000	Pass ✓
TC-02: Deposit ₹500	New balance = 1500, transaction logged	Pass ✓
TC-03: Withdraw ₹2,000 from ₹1,500 balance	Error: Insufficient funds	Pass ✓
TC-04: Delete account, then query it	404 not found returned	Pass ✓
TC-05: Retrieve transaction history	All transactions in correct order	Pass ✓
TC-06: Safe-to-Spend (budget ₹2,000, balance ₹5,000)	Safe-to-Spend = ₹3,000	Pass ✓
TC-07: Impulse classifier after 7 entertainment spends	Banner shown on 8th transaction	Pass ✓
TC-08: Transfer A → B (frontend model)	A decreases, B increases, both logged	Pass ✓
TC-09: Restart app, verify persistence	All accounts and history intact	Pass ✓
TC-10: Mobile view at 375px width	All screens render without overflow	Pass ✓

8. Limitations & Future Roadmap

Current Limitations

Limitation	Detail
No Authentication	All users share a single session; multi-user support requires JWT session management
File-based Storage	Binary flat files not suitable for concurrent multi-user access or large datasets
Transfer Command	C engine lacks an atomic TRANSFER command; transfers are modelled as two frontend transactions
Impulse Model Simplicity	Uses ratio/frequency rules, not a trained ML model; accuracy improves with more history
No Data Export	Statement views are UI-only; PDF/CSV export not yet implemented
Category Persistence	Transaction categories assigned frontend-side, not stored in the C engine data file

Future Enhancements

- Authentication & Multi-user: JWT-based login with per-user data isolation.
- Database Migration: Replace binary file storage with PostgreSQL or SQLite.
- Atomic C Transfer: Implement single TRANSFER command in the C engine.
- Category Persistence: Store transaction categories in the C engine data model.
- ML Impulse Model: Train a lightweight ONNX model on anonymised spending data.
- Analytics Dashboard: Monthly spend trends, category breakdowns, predictive charts.
- PDF Statement Export: One-click PDF generation for account statements.
- Production Deployment: Docker containerisation, HTTPS, rate limiting, environment config.

9. Conclusion

NeoBank demonstrates that a student project can do far more than meet a specification — it can challenge assumptions about what academic software is capable of.

By combining a high-performance C backend, a clean Express API bridge, and a polished React frontend, NeoBank delivers a genuinely functional banking simulation with architectural patterns that mirror production systems. More importantly, it adds two features — the Safe-to-Spend Indicator and the Impulse Purchase Classifier — that no comparable academic project currently offers.

> ★ *Financial software should help users make better decisions, not just record the ones they have already made. NeoBank shifts the paradigm from retrospective ledger to prospective advisor.*

Key Achievement	
Three-tier full-stack architecture	Cross-language communication (JS ↔ C via stdin/stdout)
Real-time predictive balance	Safe-to-Spend computed from budget + scheduled transfers
Spending behaviour classification	Actionable impulse warnings before transaction confirmation
Full CRUD banking operations	Binary file persistence via C engine — survives restarts
Production-quality UI	Dark mode, mobile layout, toast notifications, analytics

10. References & Appendix

References

1. React Documentation — https://react.dev
2. Node.js Documentation — https://nodejs.org/docs
3. Express.js Guide — https://expressjs.com/guide
4. ISO C99 Standard (ISO/IEC 9899:1999) — C Language Specification
5. MDN Web Docs — JavaScript Reference — https://developer.mozilla.org
6. Thaler, R.H. & Sunstein, C.R. (2008). Nudge. Yale University Press. [Behavioural economics foundation for impulse detection UX]
7. GNU Make Manual — https://www.gnu.org/software/make/manual
8. POSIX File I/O Reference — Open Group Base Specifications

Appendix A — Key Source Files

File	Responsibility
client/src/App.js	Main React app: routing, state, API calls, all UI screens
server/server.js	Express server: REST routes, C process manager, JSON parser
bank_backend/bank.c	C engine: structs, command parser, business rules, file I/O
bank_backend/Makefile	Compilation directives for the C banking engine
start.sh	Launcher: compile → start API → start React with health checks
bank_backend/accounts.dat	Binary data store (auto-created on first run)

Appendix B — Setup Instructions

- Clone the repository and navigate to the root directory.
- Run make in bank_backend/ to compile the C engine.
- Run npm install in both client/ and server/ directories.
- Execute ./start.sh to launch all services.
- Open http://localhost:3000 in a browser.

Appendix C — Sample API Responses

```
POST /api/accounts
→ { "id": 1042, "name": "Arjun Mehta", "type": "Savings", "balance": 5000, "created":
"2025-01-15T10:23:00Z" }

GET /api/transactions/1042
→ [{ "txn_id": 1, "type": "DEPOSIT", "amount": 2000, "balance_after": 5000,
"timestamp": "2025-01-15T11:00:00Z" }, ...]
```

End of Report — NeoBank Simulation Platform | Version 1.0